

Hot Dog POS System Architectural Document

Yina Cai

I. Kata Overview

A local hot dog stand merchant wants a point-of-sale system for his hot dog stand operators. The system needs to have sales tracking, inventory updates, social-media integration, and discount application functionalities, and it should be portable and easy to use on the street. This system also needs to support data exportation for accounting purpose.

II. Problem Statement

The goal is to design a modern, lightweight, and scalable point-of-sale system for a local hot dog stand merchant. This system will support about 50 operators and serve thousands of customers. The purpose of this system is reducing manual effort. Key constraints are time-to-market urgency and the solution needs to be feasible for at least three years, but there are no serious budget constraints.

III. Architecturally Interesting Bits

1. Functional Requirements:

- **Lightweight Operation:** The system must run on small portable devices, e.g., tables or smartphones.
- **Discounts Application:** Operators must be able to apply discounts to orders.
- **Sales Tracking:** The system must track sales data by time and location.
- **Inventory Updates:** The system needs to send inventory updates to Inventory Managers.
- **Social Media Integration:** Customers can be notified by social media app when a hot dog stand is close by.
- **Exportable Financial Data:** Operators must export data with importable formats in accounting tools from the system.

2. Non-functional Requirements:

- **Scalability:** The system must support about 50 hot dog stands and thousands of local customers, and it must scale for future business grows because this system is not expected to replace within 3 years.
- **Deployability and Testability:** Time to market is important.

Copyright (c) 2025 cyx0329.
All Rights Reserved.

- **Real-time Updates:** Immediate notifications to supply vehicles when inventory thresholds are met.
- **Resilience:** Handles connectivity interruptions in mobile environments.

3. Back-of-the-Envelope Estimations:

- About 50 hot dog stands
- An average of 100 local customers from each stand [2], which means about 5000 transactions per day - 3.5 transactions per mins.
- Each sale triggers an inventory update, which means about 5000 inventory updates per day - 3.5 updates per mins.
- During peak hours, about $50 * 40 = 2000$ orders per hour [2], 1 transaction per 2 seconds.

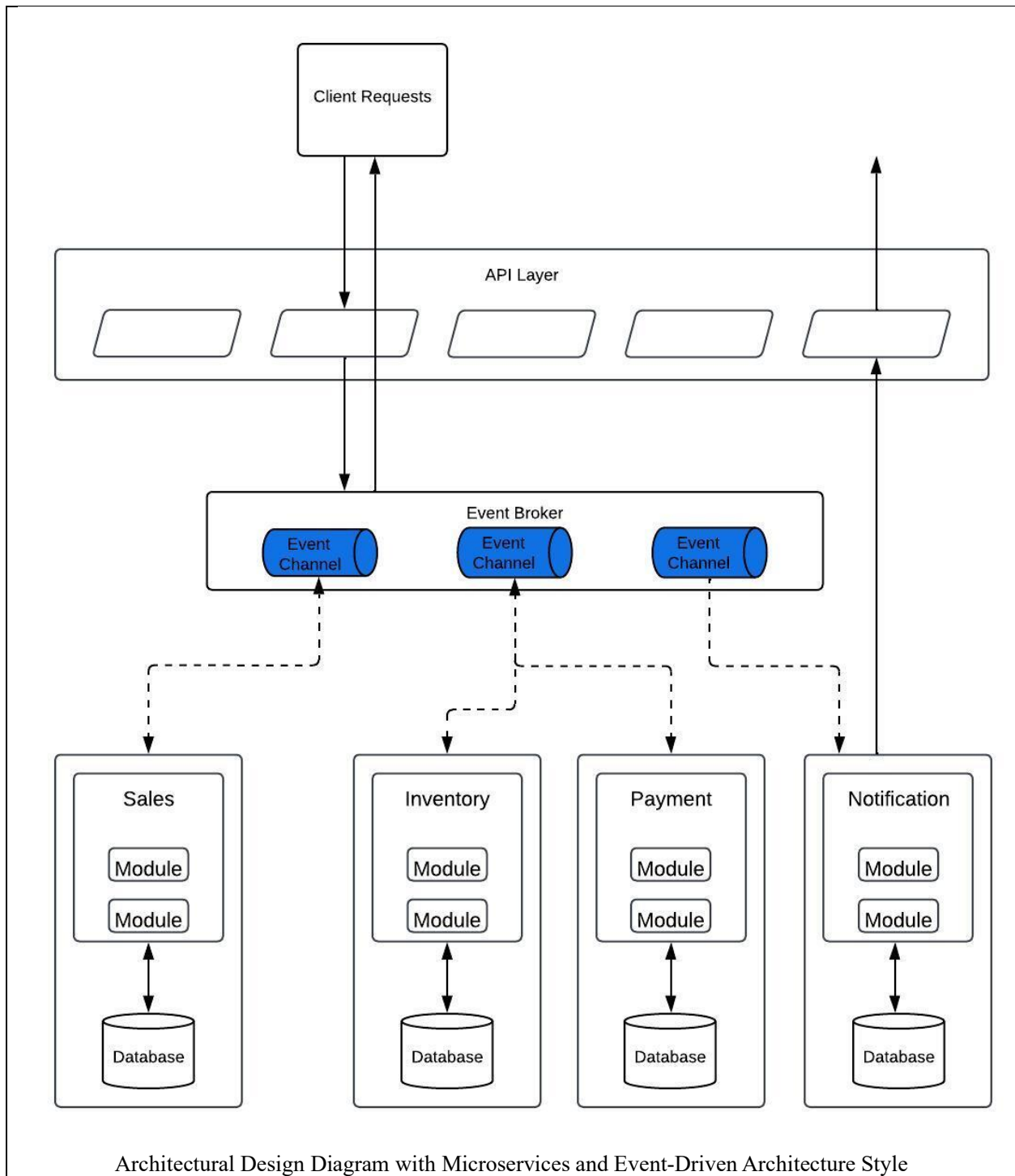
IV. Proposed Solution with a Combination of Microservices and Event-driven Architecture

A combination of microservices and event-driven architecture ensure scalability, real time updates, and elasticity. Real-time inventory updates and social media notification are functional requirement, and the system must handle dynamic spikes in activity seamlessly. Microservices and event-driven architecture provide the scalability and real-time features necessary for smooth operation.

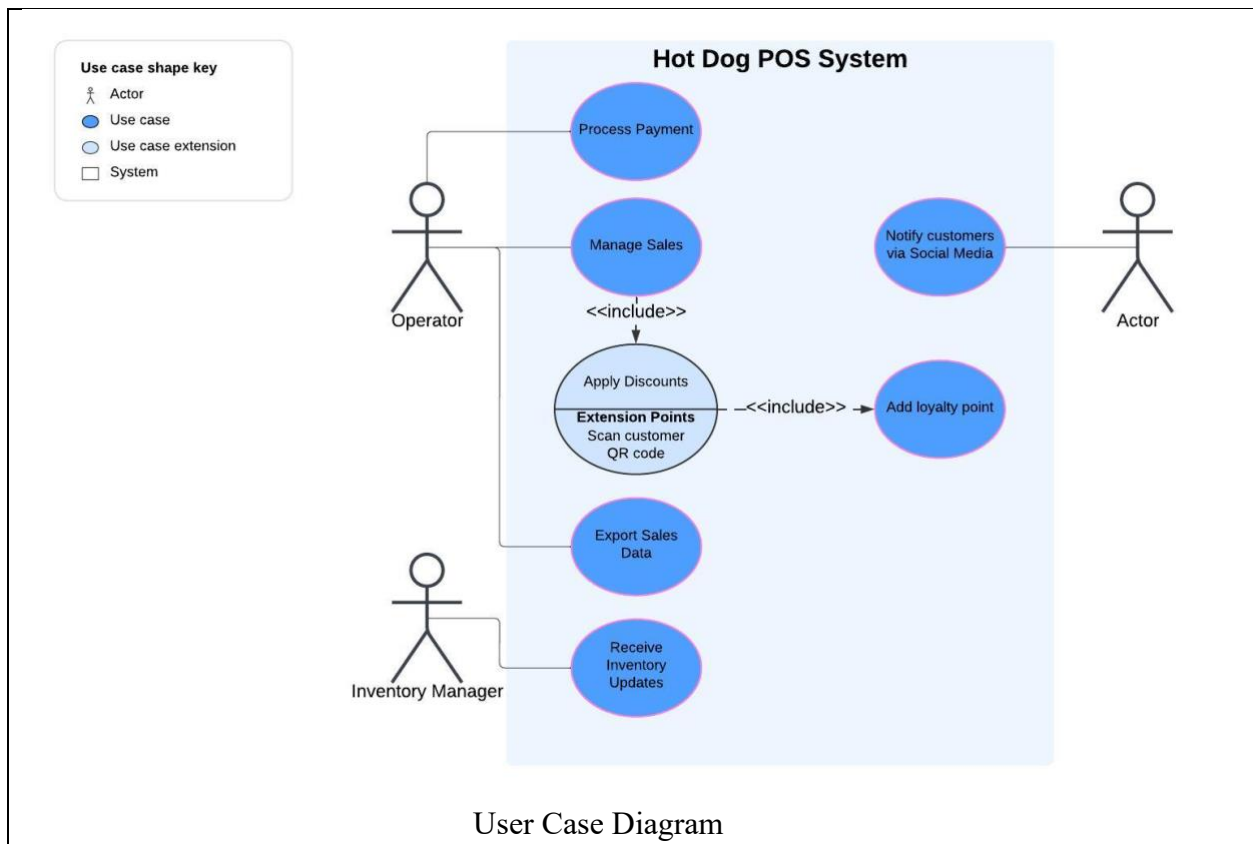
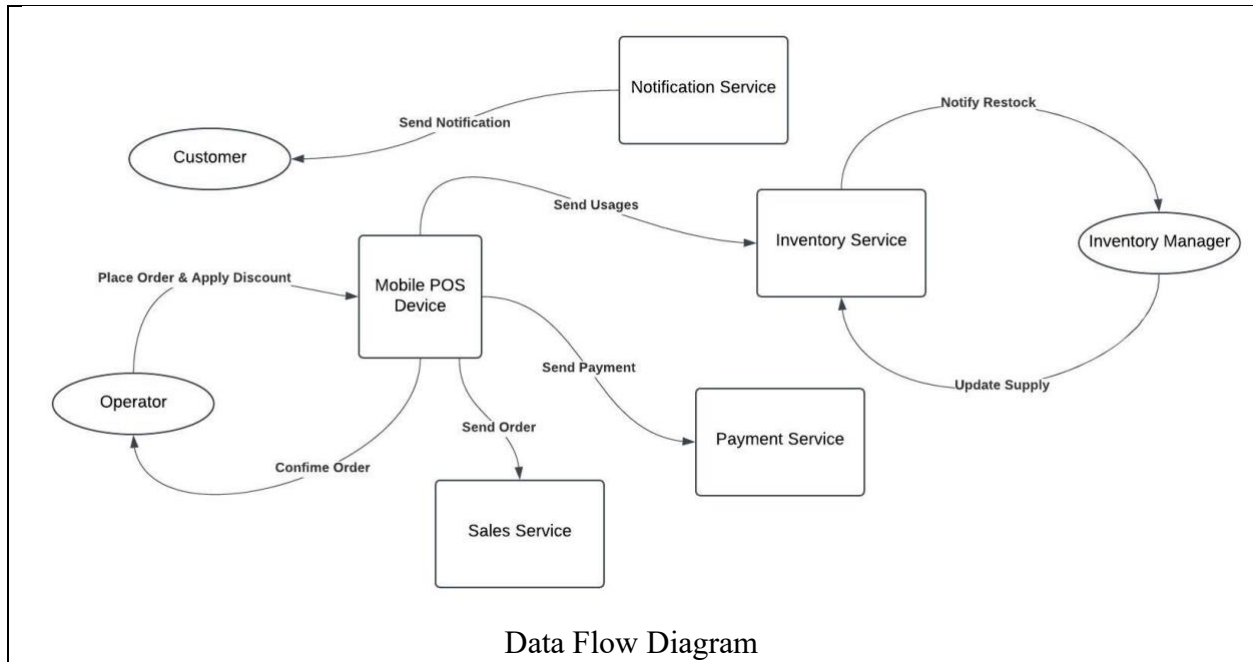
Proposed Solution with main components dives in:

- **Mobile POS Application:** Runs on tablets or smartphones to process sales, apply discounts, and update inventory. All client requests originate from this application.
- **API Layer:** Processes client requests through both internal system APIs and external social media APIs.
- **Event Broker:** Facilitates communication between API layer, sales management, inventory management, payment processing, and notification services.
- **Sales Management Service:** Handles sales transactions and applies discounts to orders. Provides sales tracking by time and location, and enables data export from the sales database.
- **Inventory Management Service:** Tracks stock levels, triggers restocking events, and update supply amounts.
- **Payment Processing Service:** Handles payment process for orders.
- **Notification Service:** Makes APIs calls to social media and send nearby notification to customers.

Copyright (c) 2025 cyx0329.
All Rights Reserved.



Copyright (c) 2025 cyx0329.
All Rights Reserved.



V. Personas

Copyright (c) 2025 cyx0329.
All Rights Reserved.

The Customer: Emily loves finding quick, convenient food options during her busy lunch breaks. She follows her favorite food vendors on social media to track their locations. Loyalty programs and discounts motivate her to become a regular customer.

The Operator: Carlos takes pride in serving customers efficiently while managing his busy stand. He relies on tools that streamline payment processing, discount application, and sales tracking.

The Inventory Manager: Linda oversees inventory distribution for all hot dog stands in the city. She depends on a system that alerts her to low supplies, helping her optimize delivery routes and maintain well-stocked stands.

VI. Solutions that Considered but Discarded

Layered Architecture – Since the time to completion is important to this system, the layered architecture popped up in my mind. The layered architecture style is a good choice for small and simple applications, but the layered architecture has very bad elasticity and scalability, which contradicts with one of the requirements that needs to be a long term 3 years solution is more important.

Microservices Architecture Alone - Without an event-driven mechanism, real-time features for example, inventory updates and social media notifications rely on synchronous calls, which may increase latency. Synchronous communication between microservices can bottleneck performance under heavy loads.

VII. Deployment Environment

Client Application: Installed and run on tables or mobile phones.

Cloud Environment: Leverage third-party cloud service for hosting microservices.

Event Broker: Managed service like AWS SNS/SQS or Kafka for message routing.

Social Media Integration: API calls to popular social media platforms.

VIII. Fitness Functions

Performance Testing: Define and validate service level agreements for expected traffic loads. Ensure 250ms response time for sales transactions.

Latency Testing: Measure response times for transaction processing and inventory updates. All services responded within an average of 250 milliseconds.

Scalability Testing: Verify consistent system performance during peak traffic periods. Support up to 200% load increase during peak hours.

Copyright (c) 2025 cyx0329.
All Rights Reserved.

Resilience: Validate operation under connectivity interruptions in mobile environments.
Social Media Integration Testing: Validate notification reliability and delivery.

IX. Failure Cases and Mitigation

Failure Cases	Mitigation	Possibility	Impact
Inventory mismatch	Implement transaction logging and conflict resolution.	Medium	High
Internet outages	Prevent internet outages data lost with offline-first pattern.	Low	High
Data loss in the cloud	Regular backups and redundancy for databases.	Low	High
API rate limits for social media	Use batching strategies and monitor API usage limits.	Medium	Medium
System overload	Auto-scale cloud infrastructure to handle increased traffic and add resources dynamically.	Medium	Medium
Service downtime	Deploy services across multiple regions with failover mechanisms.	Medium	Medium
Data loss	Use event persistence and retry mechanisms in the event broker.	Low	High

X. System States

Normal state	All services operational, transactions processed within SLAs.
Warning state	Delays in inventory updates and notification triggers.
Alarm state	Not functional and service outages.

XI. Expected Data Growth and Impacts

We expect the number of operators increase, which means transaction increase. We expect increased data storages and API usages, which requires the system to be able to scale and handle high volume traffic.

Assuming each transaction record is about 1KB, daily storage $5000 \times 1\text{KB} = 5\text{MB}$ per day, yearly story storage $5\text{MB} \times 365 = 1.8\text{ GB}$ per year.

Each transaction, inventory update and social media notification involves API calls. Total 15,000 API requests per day.

XII. Business Continuity Needs

Copyright (c) 2025 cyx0329.
All Rights Reserved.

Availability: 99.9% uptime is critical to avoid disruptions during peak hours.

Disaster Recovery: Daily backups and redundant services ensure quick recovery from outages.

Support and Maintenance: Continuous monitoring via tools like Grafana ensures proactive resolution of potential issues.

XIII. ADRs

ADR_001: Use Microservices Architecture Alone

Status:

Rejected

Context:

Initially, I considered using a microservices architecture style. However, after researching Square's architecture, I discovered they use a combination of microservices and event-driven architecture style to ensure scalability, flexibility, and reliability. This influenced my decision to change course.

Decision:

Microservices architecture presents performance challenges since distributed architectures require numerous network calls to complete operations, creating significant performance overhead [1]. Incorporating an event-driven pattern into the design addresses the communication overhead between microservices.

ADR_002: Use Event Broker Instead of Event Mediator

Status:

Rejected

Context:

After reviewing textbook page 259 Figure 17-10, I considered using orchestration with an event mediator to handle events. However, since microservices architecture emphasizes high decoupling, using an event mediator would create a central point that all events must reuse.

Decision:

This reuse introduces coupling—a negative trade-off that contradicts the primary goal of microservices [1]. To maintain decoupled communication, I decided to use an event broker instead.

Reference

[1] Mark Richards & Neal Ford, Fundamentals of Software Architecture an Engineering Approach, February 2020, First Edition, CA.

[2] CyberShu, <https://cybershu.eu/articles/arch-kata-hotdog-pos-system.html>

Copyright (c) 2025 cyx0329.
All Rights Reserved.